

Problem Formulation

Giacomo Boracchi, Luca Magri

DEIB, Politecnico di Milano

March, 31th, 2026

What is Problem Formulation

What is Problem Formulation?

A concise and formal description of the problem you are addressing in your research.

Formal?

- Yes: formal means «**with formulas.**»
- No: formal does not mean «**only formulas.**»
Text and images are needed as well!

What is Problem Formulation?

The **PF** should:

- provide a **crystal clear description of the problem you are addressing.**
- Allow the reader to identify the context you are operating in.

The **PF** should not:

- Describe your solution / the rationale behind that

Problem formulation: Key Ingredients

Key Ingredients

A good problem formulation should describe 4 key ingredients:

- The input
- The output
- Your goals / desiderata
- The «rule of the game» and challenges

The Input

A formal definition of what your algorithm processes/analyzes/modifies

The input should be specified by a

- **Notation**
- **The domain where the input lives**
- **The modeling assumptions** (Observation Model, where possible)

Example

The input in image denoising is the noisy measurement $z \in \mathbb{R}^{r \times c}$

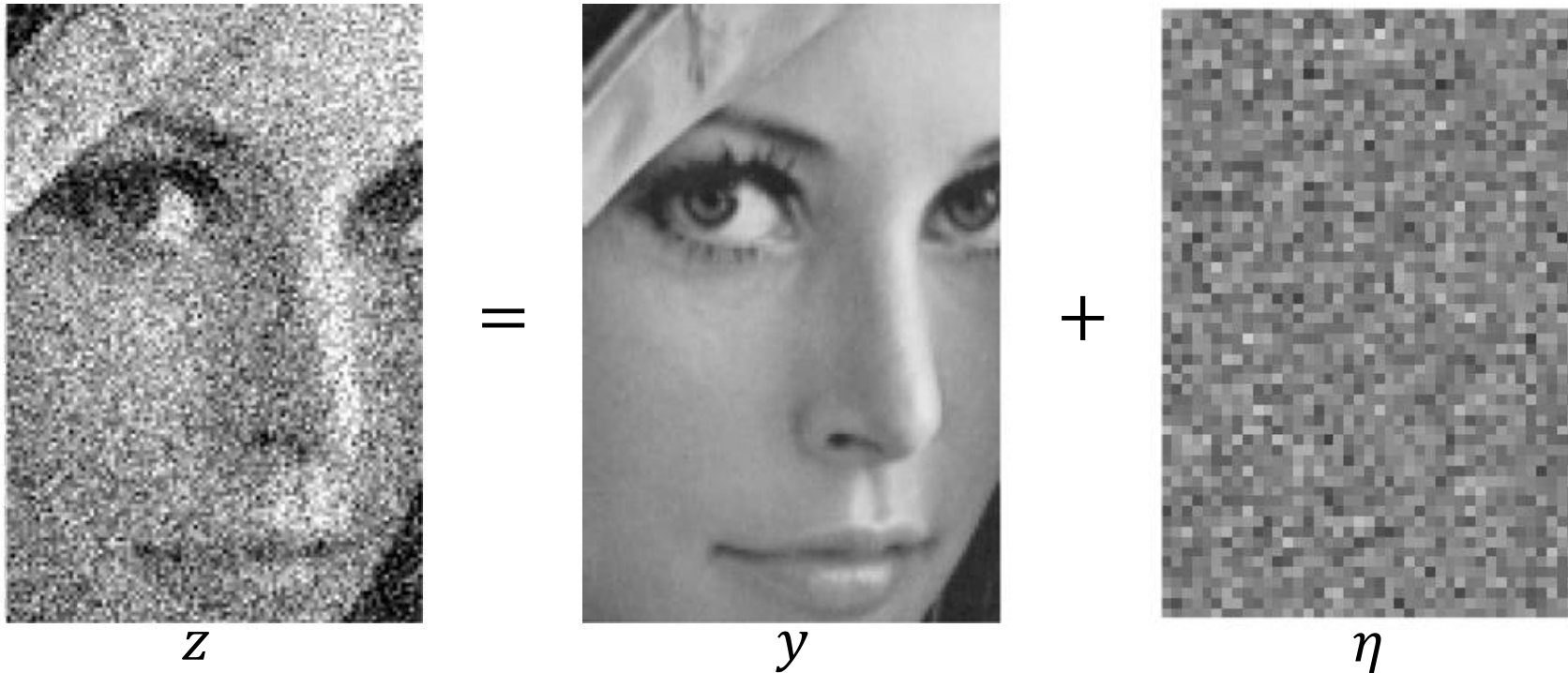
$$z(x) = y(x) + \eta(x), \quad \forall x \in X$$

Where y is the original and unknown image, and η additive white Gaussian noise, namely $\eta(x) \sim \mathcal{N}(0, \sigma^2)$

Noisy Image/Video

Noise always affects digital images

$$z(x) = y(x, t) + \eta(x), \quad x \in X, \eta \sim \mathcal{N}(0, \sigma^2)$$



Observation Model

3.1 Observation model

We consider observations as noisy grayscale images $z : X \rightarrow \mathbb{R}$ that can be modeled as

$$z(x) = y(x) + \eta(x), \quad x \in X \subset \mathbb{Z}^2, \quad (1)$$

where $X \subset \mathbb{Z}^2$ is the image domain, which is assumed a regular pixel grid, $y : X \rightarrow \mathbb{R}$ is the unknown original image, and $\eta : X \rightarrow \mathbb{R}$ is i.i.d. Gaussian white noise, $\eta(\cdot) \sim \mathcal{N}(0, \sigma^2)$.

For the sake of simplicity and notation, we assume that images can be extended beyond the boundary of X to the whole $\mathbb{Z}^2$ through any standard padding technique¹.

The Output

A formal definition of what your algorithm/solution returns

The output should be specified by a

- **Notation**
- **The domain where the output lives**

Example

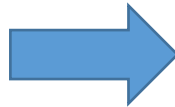
The output of image denoising is the denoised estimate $\hat{y} \in \mathbb{R}^{r \times c}$

Image/Video Denoising

The purpose of any **denoising** algorithm is to provide $\hat{y}$, an estimate of the original image y .



z



$\hat{y}$

Your goals / desiderata

This further characterizes the output but sometimes requires to be specifically discussed.

The **goal** should be specified either by:

- A quantitative criteria (e.g., minimize the reconstruction error)
- A qualitative criteria (e.g., generate a realistic image)
- Some additional notions (e.g., in change detection, you want to control false alarms)

Sometimes this is trivial and is omitted, **but you must know precisely what your goal is.**

Example:

In image denoising, the denoised estimate $\hat{y}$ should «be close to» y

The rule of the game & challenges

It is fundamental to **make apparent issues like the following:**

- What **training data** are you provided with? What annotations, if any? (*e.g. few recent training data annotated, a large number of samples with no annotation or potentially obsolete annotations*)
- What are the **specific challenges** to be addressed (*e.g. training with very small TR , adapting your model in a different conditions, limited memory*)
- Is there a **special performance measure** you aim at maximizing? (*e.g., precision over the top 100 frauds reported every day*)
- **What are you allowed to do?** How is your algorithm expected to operate? (*e.g., near real time processing, incremental learning with a single pass on your input*)

Example

In denoising you are provided with the noise variance σ^2

About «rules of the game»

Rules of the game are required to plan a **fair experimental assessment section**, which includes

- Baselines
- Competing methods
- Ideal solutions/oracles, who operate in more favorable conditions, achieving performance that you are not expected to pair

How?

How?

Problem Formulation should be in

- A short chapter / section (up to a couple of paragraphs) in your thesis,
- A dedicated slide (or couple of slides) in your presentation

Do **not** wait until thesis writing to formulate (understand???) the problem you are addressing

How?

Bear in mind that the PF also depends on the audience.

It was not required in [Krizhevsky 2012] to formulate the *plain vanilla* image classification problem to the NIPS audience, while this is necessary for AN2DL classes.

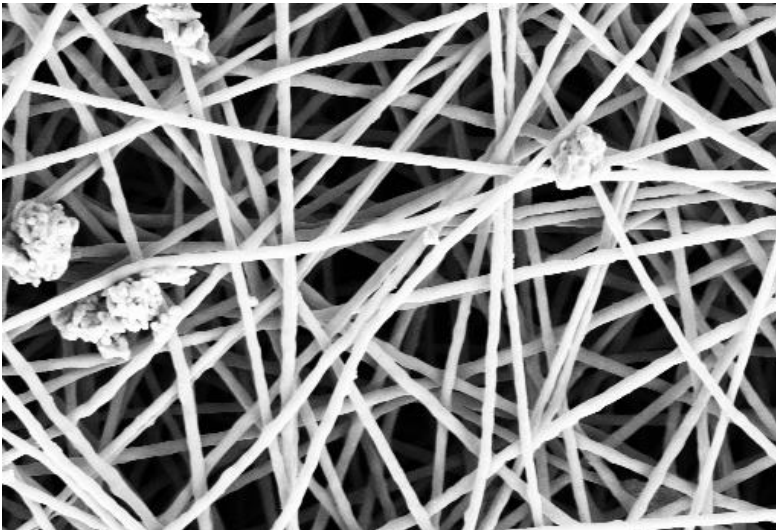
In your case, you typically address rather specific problems and have to present that to a general audience.... So PF is needed.

A Few Examples

Anomaly Detection in images

Let $s \in [0,255]^{r \times c}$ be a grayscale image defined over the pixel domain $\mathcal{X} \subset \mathbb{Z}^2$. Let $c \in \mathcal{X}$ be a pixel and $s(c)$ the corresponding intensity.

s

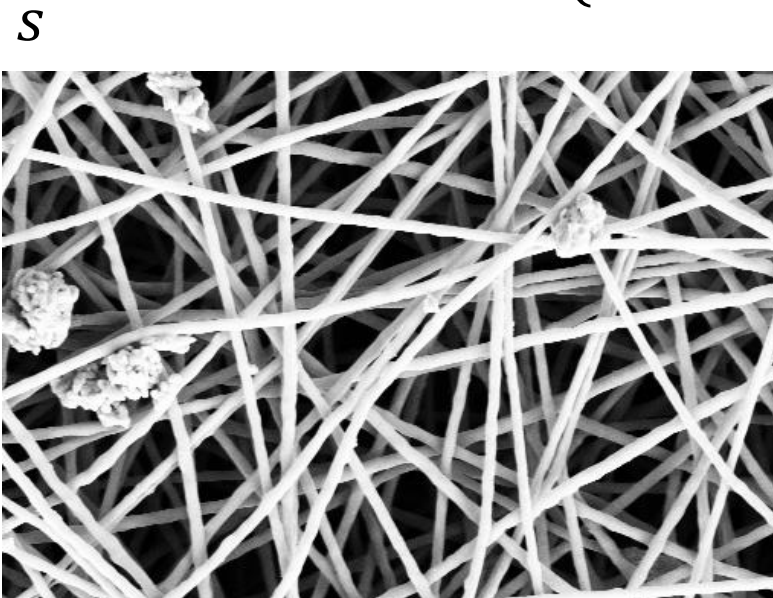


Anomaly Detection in images

Let $s \in [0,255]^{r \times c}$ be a grayscale image defined over the pixel domain $\mathcal{X} \subset \mathbb{Z}^2$. Let $c \in \mathcal{X}$ be a pixel and $s(c)$ the corresponding intensity.

Our goal is to **locate any anomalous region** in s , i.e. **estimating the unknown anomaly mask** $\Omega \in \{0,1\}^{r \times c}$ defined as

$$\Omega(c) = \begin{cases} 0 & \text{if } c \text{ falls in a normal region} \\ 1 & \text{if } c \text{ falls in an anomalous region} \end{cases}$$

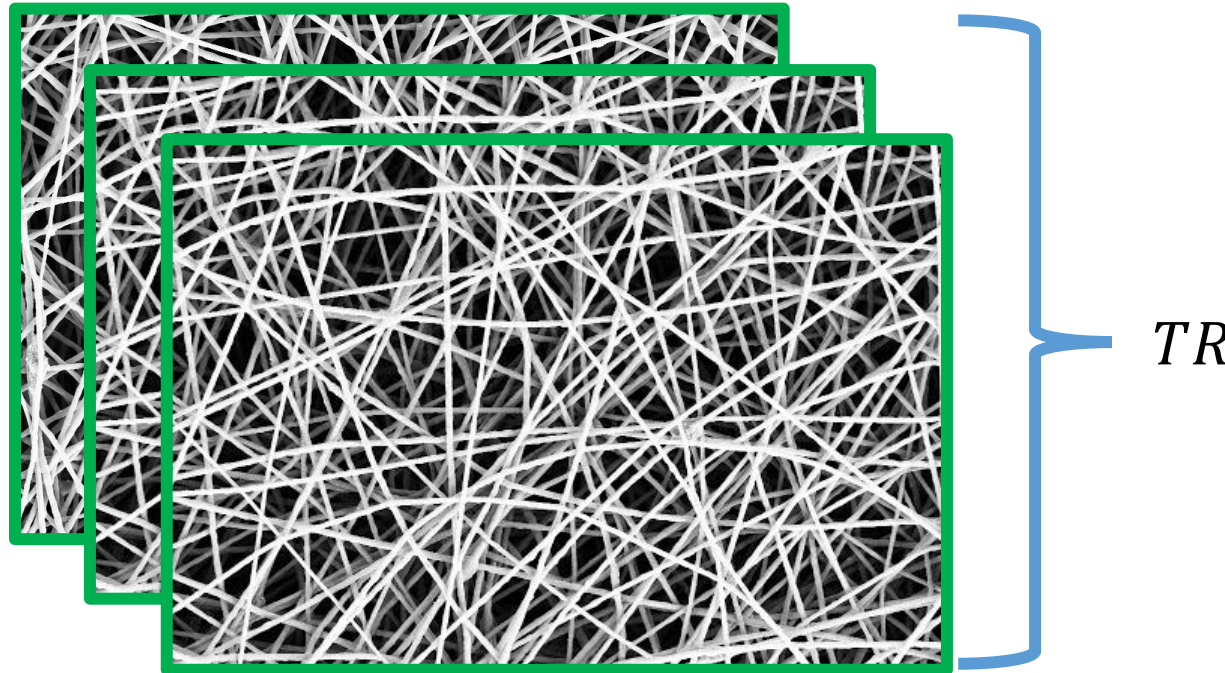


Typical Assumptions (different rules of the game)

A training set TR is provided for configuring the AD algorithm

Depending on the algorithm

- Only normal images -> semi-supervised methods
- Unlabeled images -> unsupervised methods
- Annotated images -> supervised methods



III. PROBLEM FORMULATION

Let us denote by $s : \mathcal{X} \rightarrow \mathbb{R}^+$ the SEM image depicting the nanostructures to analyze for anomaly-detection purposes, where $\mathcal{X} \subset \mathbb{Z}^2$ is the regular pixel grid corresponding to the image domain. The image intensity² at pixel $c \in \mathcal{X}$ is denoted by $s(c)$. Our goal is to locate anomalous regions in s ; as such, the problem can be formulated as estimating the unknown anomaly mask

$$\Omega(c) = \begin{cases} 0 & \text{if } c \text{ falls inside a normal region} \\ 1 & \text{if } c \text{ falls inside an anomalous region} \end{cases} \quad (1)$$

which has to report as many anomalies as possible, independently of their dimension and shape. In particular, we are interested in estimating an anomaly mask $\tilde{\Omega}$ that *i*) covers most of the anomalous regions in s and that *ii*) reports the largest number of anomalies, including the smallest ones.

Our only assumption is that, for training purposes, a set of normal (i.e., anomaly-free) images $\mathcal{T}$ is given, while no training images containing anomalous regions are provided. This is a reasonable assumption since anomalous regions might be very different in shape, dimension and appearance, and a training set might not encompass all possible anomalies that could occur during operations. In this sense, anomalies remain unknown and we detect as anomalous any region that do not conform the structure of normal data.

2 Problem formulation

We consider a 3D scene containing M cylinders $\mathcal{C}_j$ in general position, each with known position, axis direction $\mathbf{w}_j$, and radius. A camera with an unknown but fixed intrinsic matrix $\mathbf{K}$ captures N images $\{I_1, \dots, I_N\}$ of the scene. Each image I_i is associated with an unknown camera matrix $\mathbf{P}_i = \mathbf{K}[\mathbf{R}_i | \mathbf{t}_i]$, where $\mathbf{R}_i$ and $\mathbf{t}_i$ denote the rotation and translation of the camera in view i . Each cylinder $\mathcal{C}_j$ projects onto each view I_i as a pair of *silhouette lines*, denoted l_{ij}^h with $h = 1, 2$ (see Figure 2 that displays one of the two silhouette lines of $\mathcal{C}_j$). These lines correspond to the visible contours of the cylindrical surface. Each l_{ij}^h pair arises from the intersection between the image plane and the planes Π_{ij}^h that pass through the optical center of the camera $\mathbf{P}_i$ and are tangent to the surface of $\mathcal{C}_j$.

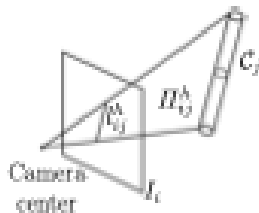


Fig. 2: A cylinder $\mathcal{C}_j$ and one of its silhouette line l_{ij}^h

Given in input the parameters of the 3D cylinders $\{\mathcal{C}_j\}_{j=1}^M$ and a collection of silhouette lines $\{l_{ij}^h\}$ observed in the images, our aim is twofold:

1. **Camera calibration:** estimate the intrinsic parameters of the calibration matrix

$$\mathbf{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad (1)$$

that correspond to the focal lengths f_x, f_y , the skew s , and the coordinates of the principal point (c_x, c_y) . Depending on the assumptions made on $\mathbf{K}$, different variants of the problem can be considered. By introducing suitable linear constraints, such as zero skew ($s = 0$), known aspect ratio ($f_x = \alpha f_y$), or a centered principal point, the original set of five unknowns can be reduced.

2. **Pose estimation:** estimate the extrinsic parameters, namely rotations $\mathbf{R}_i$ and translations $\mathbf{t}_i$, for each image I_i .

Pose from cylinders

Context

2 Problem formulation

We consider a 3D scene containing M cylinders C_j in general position, each with known position, axis direction $\mathbf{w}_j$, and radius. A camera with an unknown but fixed intrinsic matrix $\mathbf{K}$ captures N images $\{I_1, \dots, I_N\}$ of the scene. Each image I_i is associated with an unknown camera matrix $\mathbf{P}_i = \mathbf{K}[\mathbf{R}_i | \mathbf{t}_i]$, where $\mathbf{R}_i$ and $\mathbf{t}_i$ denote the rotation and translation of the camera in view i . Each cylinder C_j projects onto each view I_i as a pair of *silhouette lines*, denoted l_{ij}^h with $h = 1, 2$ (see Figure 2 that displays one of the two silhouette lines of C_j). These lines correspond to the visible contours of the cylindrical surface. Each l_{ij}^h pair arises from the intersection between the image plane and the planes Π_{ij}^h that pass through the optical center of the camera $\mathbf{P}_i$ and are tangent to the surface of C_j .

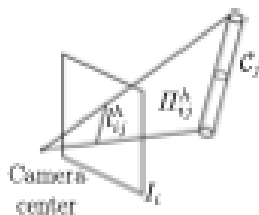


Fig. 2: A cylinder C_j and one of its silhouette line l_{ij}^h

Given in input the parameters of the 3D cylinders $\{C_j\}_{j=1}^M$ and a collection of silhouette lines $\{l_{ij}^h\}$ observed in the images, our aim is twofold:

1. **Camera calibration:** estimate the intrinsic parameters of the calibration matrix

$$\mathbf{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad (1)$$

that correspond to the focal lengths f_x, f_y , the skew s , and the coordinates of the principal point (c_x, c_y) . Depending on the assumptions made on $\mathbf{K}$, different variants of the problem can be considered. By introducing suitable linear constraints, such as zero skew ($s = 0$), known aspect ratio ($f_x = \alpha f_y$), or a centered principal point, the original set of five unknowns can be reduced.

2. **Pose estimation:** estimate the extrinsic parameters, namely rotations $\mathbf{R}_i$ and translations $\mathbf{t}_i$, for each image I_i .

Pose from cylinders

Input

2 Problem formulation

We consider a 3D scene containing M cylinders C_j in general position, each with known position, axis direction $\mathbf{w}_j$, and radius. A camera with an unknown but fixed intrinsic matrix $\mathbf{K}$ captures N images $\{I_1, \dots, I_N\}$ of the scene. Each image I_i is associated with an unknown camera matrix $\mathbf{P}_i = \mathbf{K}[\mathbf{R}_i | \mathbf{t}_i]$, where $\mathbf{R}_i$ and $\mathbf{t}_i$ denote the rotation and translation of the camera in view i . Each cylinder C_j projects onto each view I_i as a pair of *silhouette lines*, denoted l_{ij}^h with $h = 1, 2$ (see Figure 2 that displays one of the two silhouette lines of C_j). These lines correspond to the visible contours of the cylindrical surface. Each l_{ij}^h pair arises from the intersection between the image plane and the planes Π_{ij}^h that pass through the optical center of the camera $\mathbf{P}_i$ and are tangent to the surface of C_j .

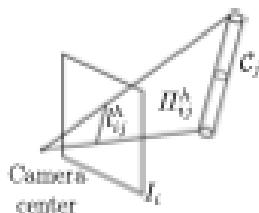


Fig. 2: A cylinder C_j and one of its silhouette line l_{ij}^h

Given in input the parameters of the 3D cylinders $\{C_j\}_{j=1}^M$ and a collection of silhouette lines $\{l_{ij}^h\}$ observed in the images, our aim is twofold:

1. **Camera calibration:** estimate the intrinsic parameters of the calibration matrix

$$\mathbf{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad (1)$$

that correspond to the focal lengths f_x, f_y , the skew s , and the coordinates of the principal point (c_x, c_y) . Depending on the assumptions made on $\mathbf{K}$, different variants of the problem can be considered. By introducing suitable linear constraints, such as zero skew ($s = 0$), known aspect ratio ($f_x = \alpha f_y$), or a centered principal point, the original set of five unknowns can be reduced.

2. **Pose estimation:** estimate the extrinsic parameters, namely rotations $\mathbf{R}_i$ and translations $\mathbf{t}_i$, for each image I_i .

Pose from cylinders

Output (in this case two output)

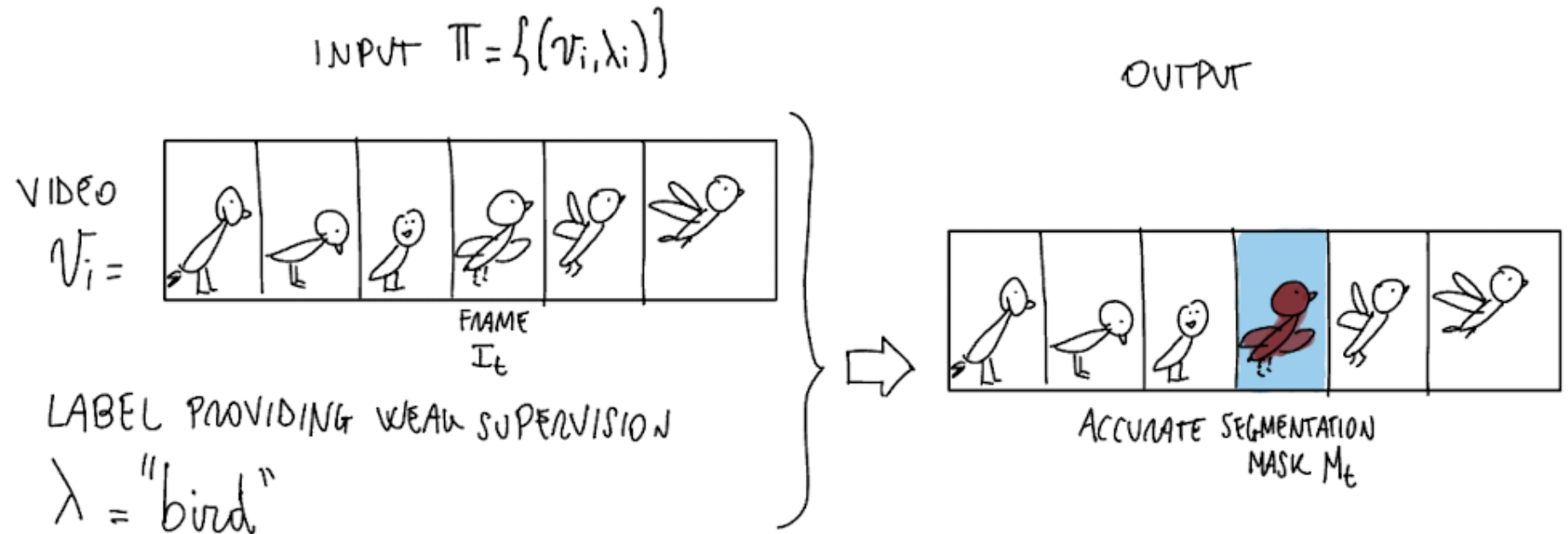
Video semantic segmentation

2 Problem Formulation

WS-VSS aims to learn temporally consistent pixel-level segmentation mask for each frame in a video using only coarse annotations available at video level. Formally, given a training set $\mathbb{T} = \{(v_i, y_i)\}_{i=1}^N$, where each video $v_i = \{I_t \in \mathbb{R}^{H \times W \times 3}\}_{t=1}^{T_i}$ is associated with a single video-level label $y_i \in \{1, \dots, C\}$, our goal is to learn a function F_θ that generates a dense segmentation mask $M_t = F_\theta(I_t) \in [0, 1]^{H \times W \times |C|}$ for each frame I_t . Predictions must satisfy two properties: (i) they should be semantically aligned with the video-level label y_i , and (ii) they should remain temporally consistent across consecutive frames within v_i .

Rules of the game

You can start with a cartoon illustration to begin with... you have time to refine it



Video semantic segmentation

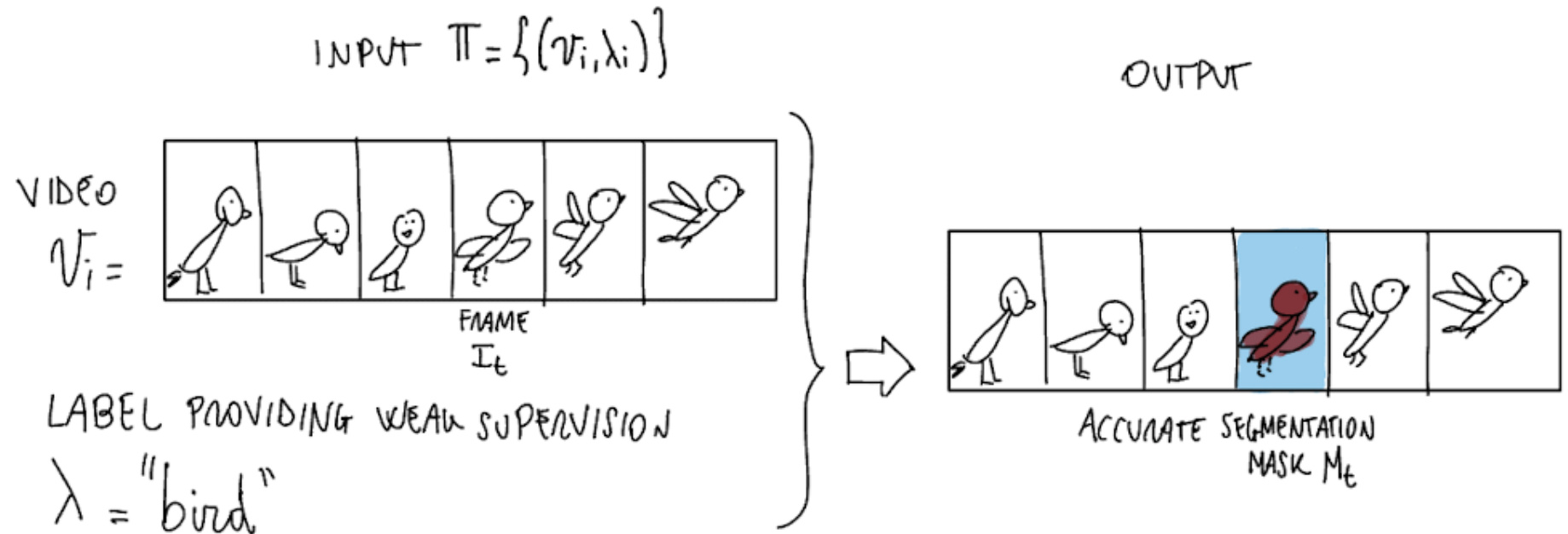
2 Problem Formulation

WS-VSS aims to learn temporally consistent pixel-level segmentation mask for each frame in a video using only coarse annotations available at video level.

Formally, given a training set $\mathbb{T} = \{(v_i, y_i)\}_{i=1}^N$, where each video $v_i = \{I_t \in \mathbb{R}^{H \times W \times 3}\}_{t=1}^{T_i}$ is associated with a single video-level label $y_i \in \{1, \dots, C\}$, our goal is to learn a function F_θ that generates a dense segmentation mask $M_t = F_\theta(I_t) \in [0, 1]^{H \times W \times |C|}$ for each frame I_t . Predictions must satisfy two properties: (i) they should be semantically aligned with the video-level label y_i , and (ii) they should remain temporally consistent across consecutive frames within v_i .

Input

You can start with a cartoon illustration to begin with... you have time to refine it



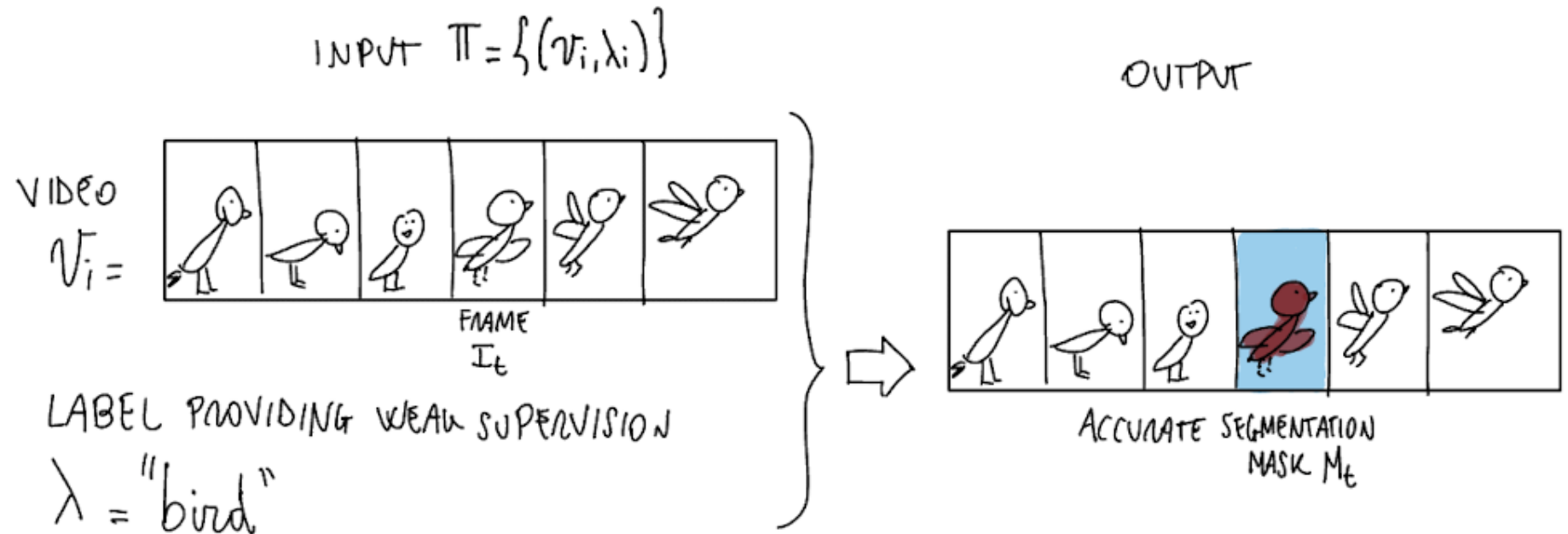
Video semantic segmentation

2 Problem Formulation

WS-VSS aims to learn temporally consistent pixel-level segmentation mask for each frame in a video using only coarse annotations available at video level. Formally, given a training set $\mathbb{T} = \{(v_i, y_i)\}_{i=1}^N$, where each video $v_i = \{I_t \in \mathbb{R}^{H \times W \times 3}\}_{t=1}^{T_i}$ is associated with a single video-level label $y_i \in \{1, \dots, C\}$, our goal is to learn a function F_θ that generates a dense segmentation mask $M_t = F_\theta(I_t) \in [0, 1]^{H \times W \times |C|}$ for each frame I_t . Predictions must satisfy two properties: (i) they should be semantically aligned with the video-level label y_i , and (ii) they should remain temporally consistent across consecutive frames within v_i .

Output

You can start with a cartoon illustration to begin with... you have time to refine it



Explainable clustering

2 Problem formulation

The problem of making a clustering explainable can be framed as follows. Let $\mathcal{X} \subseteq \mathbb{R}^d$ be a dataset with $|\mathcal{X}| = n$ data points. We are given as input a *reference clustering*, namely a partition of $\mathcal{X}$ in k clusters denoted as $\mathcal{C} = \{C_1, \dots, C_k\} \subseteq \mathcal{P}(\mathcal{X})$. A typical example of input is depicted in Figure 2a, where a color-coded clustering obtained by k -Means with $k = 3$ is represented.

The aim of explainable clustering is to produce a novel clustering $\hat{\mathcal{C}} = \{\hat{C}_1, \dots, \hat{C}_k\} \subseteq \mathcal{P}(\mathcal{X})$ that is more explainable and “as close as possible” to the reference clustering $\mathcal{C}$. An example is reported on the right of Figure 2b where an explainable clustering $\hat{\mathcal{C}}$ is obtained by partitioning the input space into a collection of 3 rectangles defined by simple binary conditions. For instance, we can explain why a point $\mathbf{x} = (x_1, x_2)$ is assigned to cluster 1 (in green) with a sequence of two binary conditions $\mathcal{S}(\mathbf{x}) = \{\{x_2 > t_2\}, \{x_1 \leq t_1\}\}$ using axis-parallel cuts. In general, each cluster assignment of $\hat{\mathcal{C}}$ can be explained by a sequence of these conditions and the explanation complexity is measured by $\mathcal{S}(\hat{\mathcal{C}}) = \max_{\mathbf{x} \in \mathcal{X}} |\mathcal{S}(\mathbf{x})|$, that is the length of the longest sequence $\mathcal{S}(\mathbf{x})$ over all $\mathbf{x} \in \mathcal{X}$. It follows that the output of an explainable clustering algorithm is a partition $\hat{\mathcal{R}} = \{\hat{R}_1, \dots, \hat{R}_k\} \subseteq \mathcal{P}(\mathbb{R}^d)$, defined by the union of axis-parallel hyper-rectangles, that induces an explainable clustering $\hat{\mathcal{C}}$ over points $\mathcal{X}$.

The explainable clustering $\hat{\mathcal{C}}$ minimizes a specific function *cost* : $\mathcal{P}(\mathcal{X}) \rightarrow \mathbb{R}$ to approximate the reference clustering $\mathcal{C}$. An example of *cost* function is the number of misclassified points with respect to $\mathcal{C}$, represented by the red crosses in Figure 2b. Indeed, imposing that clusters reside in the union of axis-parallel hyper-rectangular cells R_c with $c \in \{1, \dots, k\}$ inevitably results in some misclassification errors, due to the trade-off between accuracy and explainability [1].

Input

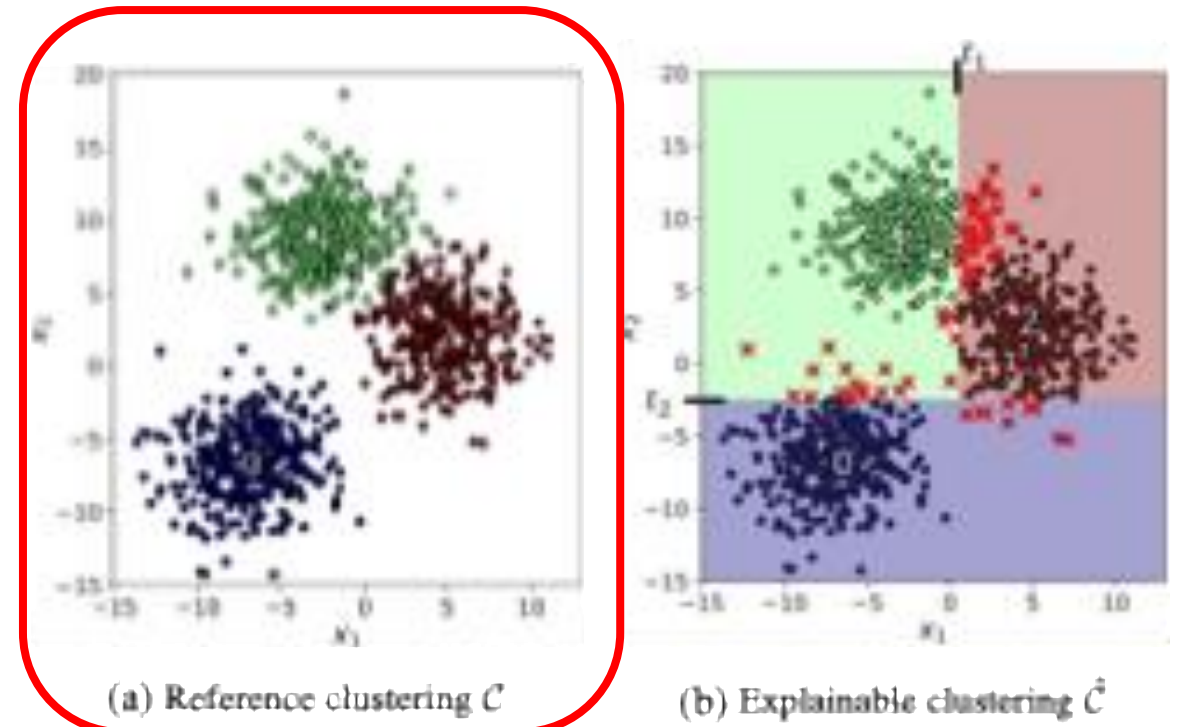


Figure 2: Making a reference clustering (a) explainable (b): points are assigned to clusters using a sequence of binary conditions based on feature-threshold pairs (x_i, t_i) . The accuracy-explainability trade-off is paid in terms of misclassified points marked as red crosses $\times$.

Explainable clustering

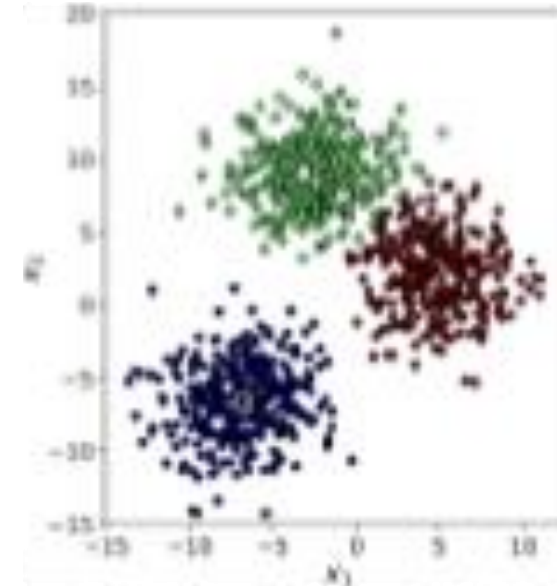
2 Problem formulation

The problem of making a clustering explainable can be framed as follows. Let $\mathcal{X} \subseteq \mathbb{R}^d$ be a dataset with $|\mathcal{X}| = n$ data points. We are given as input a *reference clustering*, namely a partition of $\mathcal{X}$ in k clusters denoted as $\mathcal{C} = \{C_1, \dots, C_k\} \subseteq \mathcal{P}(\mathcal{X})$. A typical example of input is depicted in Figure 2a, where a color-coded clustering obtained by k -Means with $k = 3$ is represented.

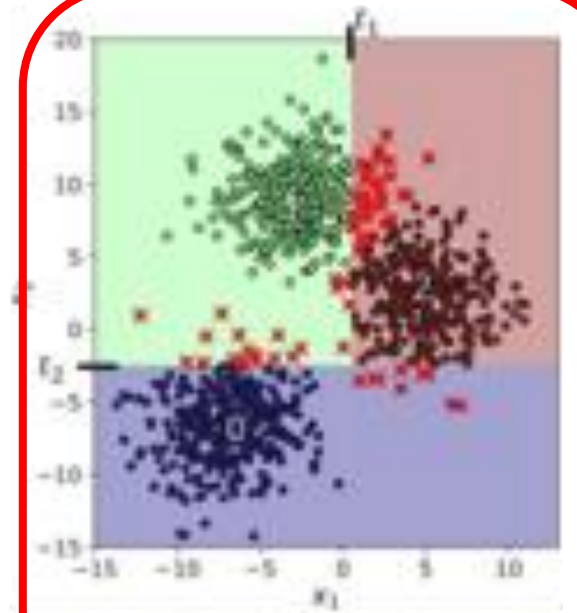
The aim of explainable clustering is to produce a novel clustering $\hat{\mathcal{C}} = \{\hat{C}_1, \dots, \hat{C}_k\} \subseteq \mathcal{P}(\mathcal{X})$ that is more explainable and “as close as possible” to the reference clustering $\mathcal{C}$. An example is reported on the right of Figure 2b where an explainable clustering $\hat{\mathcal{C}}$ is obtained by partitioning the input space into a collection of 3 rectangles defined by simple binary conditions. For instance, we can explain why a point $\mathbf{x} = (x_1, x_2)$ is assigned to cluster 1 (in green) with a sequence of two binary conditions $\mathcal{S}(\mathbf{x}) = \{\{x_2 > t_2\}, \{x_1 \leq t_1\}\}$ using axis-parallel cuts. In general, each cluster assignment of $\hat{\mathcal{C}}$ can be explained by a sequence of these conditions and the explanation complexity is measured by $\mathcal{S}(\hat{\mathcal{C}}) = \max_{\mathbf{x} \in \mathcal{X}} |\mathcal{S}(\mathbf{x})|$, that is the length of the longest sequence $\mathcal{S}(\mathbf{x})$ over all $\mathbf{x} \in \mathcal{X}$. It follows that the output of an explainable clustering algorithm is a partition $\hat{\mathcal{R}} = \{\hat{R}_1, \dots, \hat{R}_k\} \subseteq \mathcal{P}(\mathbb{R}^d)$, defined by the union of axis-parallel hyper-rectangles, that induces an explainable clustering $\hat{\mathcal{C}}$ over points $\mathcal{X}$.

The explainable clustering $\hat{\mathcal{C}}$ minimizes a specific function *cost* : $\mathcal{P}(\mathcal{X}) \rightarrow \mathbb{R}$ to approximate the reference clustering $\mathcal{C}$. An example of *cost* function is the number of misclassified points with respect to $\mathcal{C}$, represented by the red crosses in Figure 2b. Indeed, imposing that clusters reside in the union of axis-parallel hyper-rectangular cells R_c with $c \in \{1, \dots, k\}$ inevitably results in some misclassification errors, due to the trade-off between accuracy and explainability [1].

Output



(a) Reference clustering $\mathcal{C}$



(b) Explainable clustering $\hat{\mathcal{C}}$

Figure 2: Making a reference clustering (a) explainable (b): points are assigned to clusters using a sequence of binary conditions based on feature-threshold pairs (x_i, t_i) . The accuracy-explainability trade-off is paid in terms of misclassified points marked as red crosses $\times$.

Explainable clustering

2 Problem formulation

The problem of making a clustering explainable can be framed as follows. Let $\mathcal{X} \subseteq \mathbb{R}^d$ be a dataset with $|\mathcal{X}| = n$ data points. We are given as input a *reference clustering*, namely a partition of $\mathcal{X}$ in k clusters denoted as $\mathcal{C} = \{C_1, \dots, C_k\} \subseteq \mathcal{P}(\mathcal{X})$. A typical example of input is depicted in Figure 2a, where a color-coded clustering obtained by k -Means with $k = 3$ is represented.

The aim of explainable clustering is to produce a novel clustering $\hat{\mathcal{C}} = \{\hat{C}_1, \dots, \hat{C}_k\} \subseteq \mathcal{P}(\mathcal{X})$ that is more explainable and “as close as possible” to the reference clustering $\mathcal{C}$. An example is reported on the right of Figure 2b where an explainable clustering $\hat{\mathcal{C}}$ is obtained by partitioning the input space into a collection of 3 rectangles defined by simple binary conditions. For instance, we can explain why a point $\mathbf{x} = (x_1, x_2)$ is assigned to cluster 1 (in green) with a sequence of two binary conditions $\mathcal{S}(\mathbf{x}) = \{\{x_2 > t_2\}, \{x_1 \leq t_1\}\}$ using axis-parallel cuts. In general, each cluster assignment of $\hat{\mathcal{C}}$ can be explained by a sequence of these conditions and the explanation complexity is measured by $\mathcal{S}(\hat{\mathcal{C}}) = \max_{\mathbf{x} \in \mathcal{X}} |\mathcal{S}(\mathbf{x})|$, that is the length of the longest sequence $\mathcal{S}(\mathbf{x})$ over all $\mathbf{x} \in \mathcal{X}$. It follows that the output of an explainable clustering algorithm is a partition $\hat{\mathcal{R}} = \{\hat{R}_1, \dots, \hat{R}_k\} \subseteq \mathcal{P}(\mathbb{R}^d)$, defined by the union of axis-parallel hyper-rectangles, that induces an explainable clustering $\hat{\mathcal{C}}$ over points $\mathcal{X}$.

The explainable clustering $\hat{\mathcal{C}}$ minimizes a specific function *cost* : $\mathcal{P}(\mathcal{X}) \rightarrow \mathbb{R}$ to approximate the reference clustering $\mathcal{C}$. An example of *cost* function is the number of misclassified points with respect to $\mathcal{C}$, represented by the red crosses in Figure 2b. Indeed, imposing that clusters reside in the union of axis-parallel hyper-rectangular cells R_c with $c \in \{1, \dots, k\}$ inevitably results in some misclassification errors, due to the trade-off between accuracy and explainability [1].

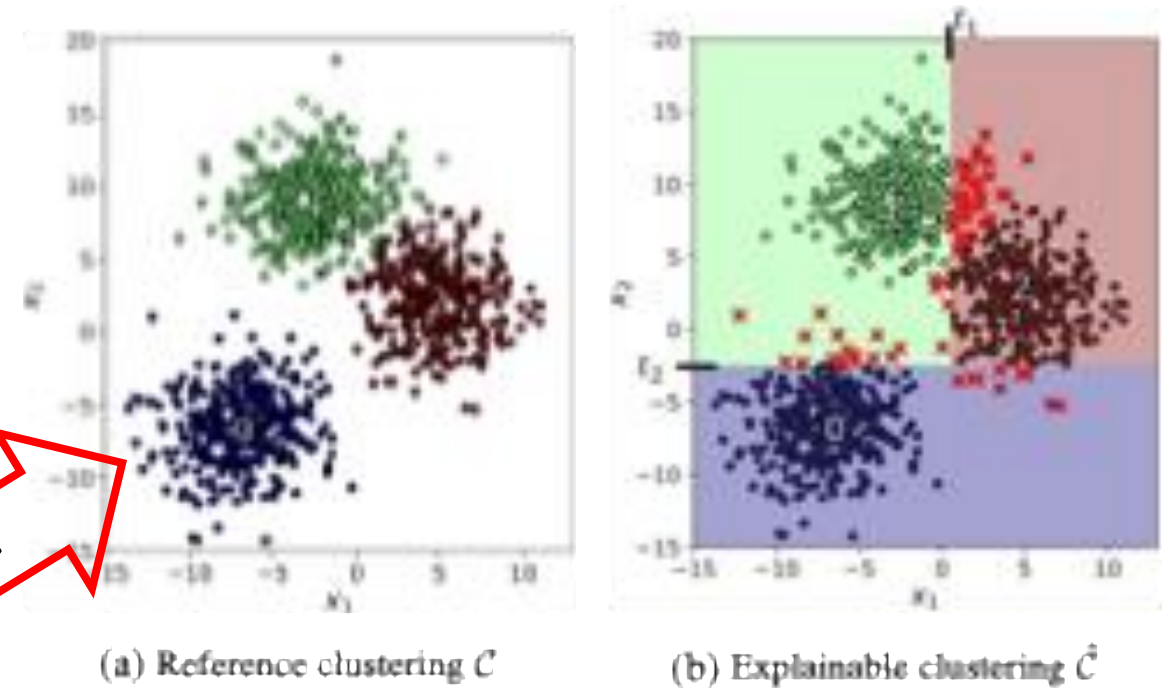


Figure 2: Making a reference clustering (a) explainable (b): points are assigned to clusters using a sequence of binary conditions based on feature-threshold pairs (x_i, t_i) . The accuracy-explainability trade-off is paid in terms of misclassified points marked as red crosses $\times$.

Note that the figure is described in the text and it is used to make the problem more accessible

Change-Detection in a statistical framework

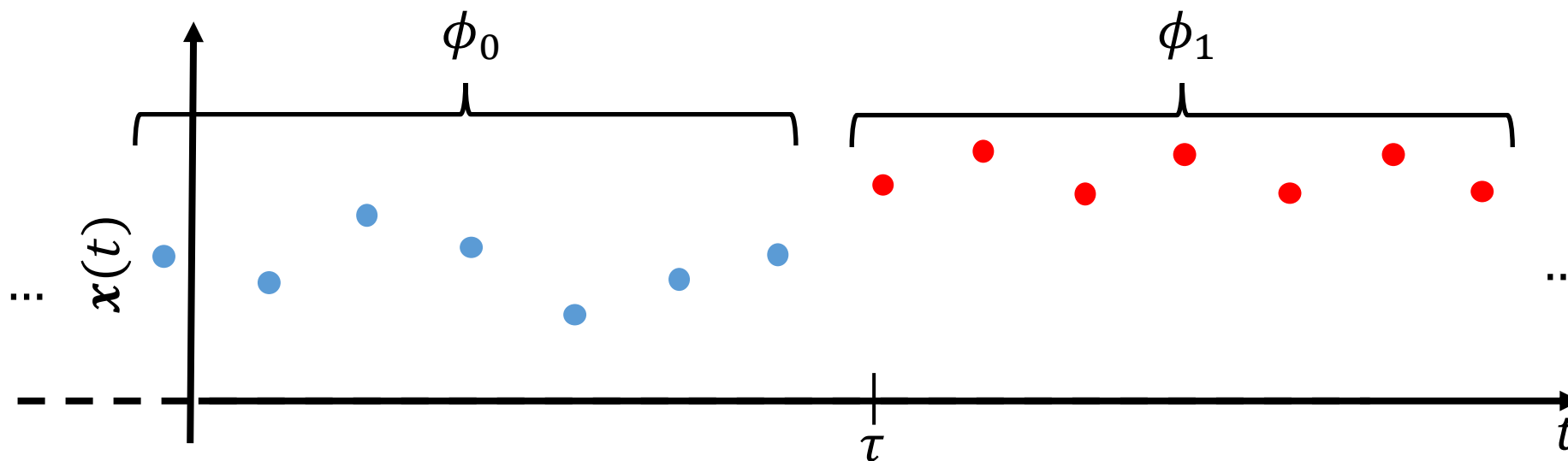
Often, the change-detection problem **boils down to**:

Monitor a **stream** $\{x(t), t = 1, \dots\}$, $x(t) \in \mathbb{R}^d$ of realizations of a **random variable**, and **detect the change-point** τ ,

$$x(t) \sim \begin{cases} \phi_0 & t < \tau \\ \phi_1 & t \geq \tau \end{cases} \quad \begin{array}{l} \text{in control state} \\ \text{out of control state} \end{array},$$

where $\{x(t), t < \tau\}$ **are i.i.d.** and $\phi_0 \neq \phi_1$

We denote such change as: $\phi_0 \rightarrow \phi_1$



Change-Detection in a statistical framework

Often, the change-detection problem **boils down to**:

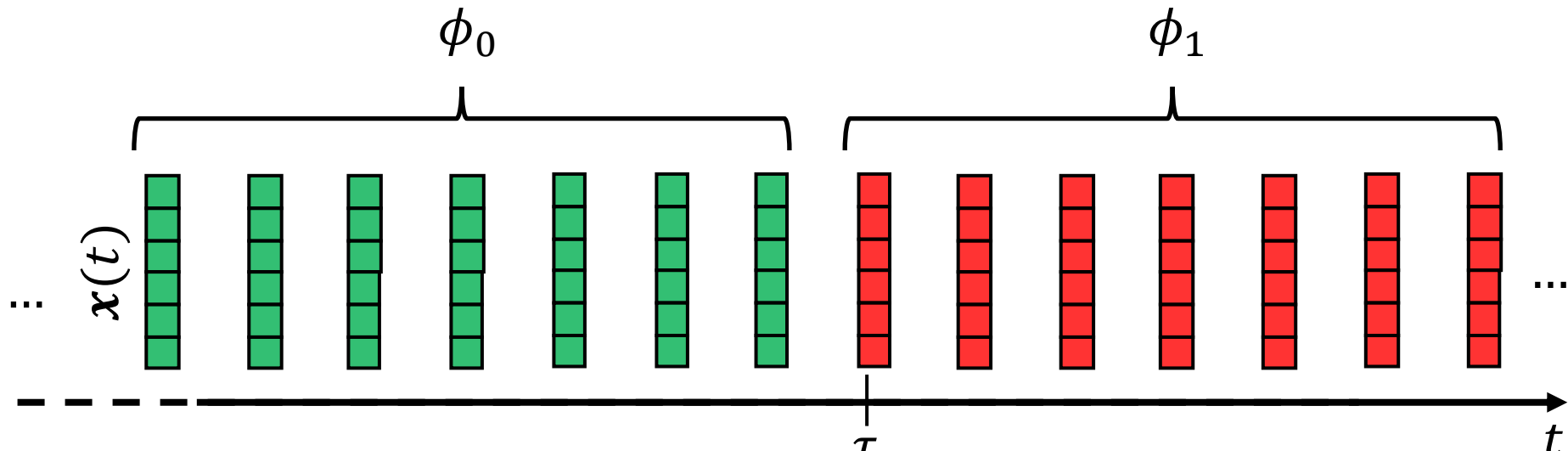
Monitor a **stream** $\{x(t), t = 1, \dots\}$, $x(t) \in \mathbb{R}^d$ of realizations of a **random variable**, and **detect the change-point** τ ,

$$x(t) \sim \begin{cases} \phi_0 & t < \tau \\ \phi_1 & t \geq \tau \end{cases} \quad \begin{array}{l} \text{in control state} \\ \text{out of control state} \end{array},$$

where $\{x(t), t < \tau\}$ **are i.i.d.** and $\phi_0 \neq \phi_1$

We denote such change as: $\phi_0 \rightarrow \phi_1$

And in particular we are interested to **multivariate settings**: $d > 1$



3 Problem Formulation

We address the online change-detection problem in a virtually unlimited multivariate datastream $x_1, x_2 \dots \in \mathbb{R}^d$. We assume that, as long as there are no changes, all the data samples are i.i.d. realizations of a random variable having unknown distribution ϕ_0 . We define the *change point* τ as the unknown time instant when a change $\phi_0 \rightarrow \phi_1$ takes place:

$$x_t \sim \begin{cases} \phi_0 & \text{if } t < \tau \\ \phi_1 & \text{if } t \geq \tau \end{cases} . \quad (1)$$

We assume that both ϕ_0 and $\phi_1 \neq \phi_0$ are unknown, and that a training set TR containing N realizations of ϕ_0 is provided.

Online change-detection algorithms assess, for each new incoming sample x_t , whether the sequence $\{x_1, \dots, x_t\}$ contains a change point. Typically, a statistic T_t is computed at each incoming x_t , then a decision rule is applied. Usually, the rule consists in controlling whether $T_t > h_t$ for an appropriate threshold h_t , and the detection time t^* is defined as the first time index when this happens:

$$t^* = \min\{t : T_t > h_t\}. \quad (2)$$

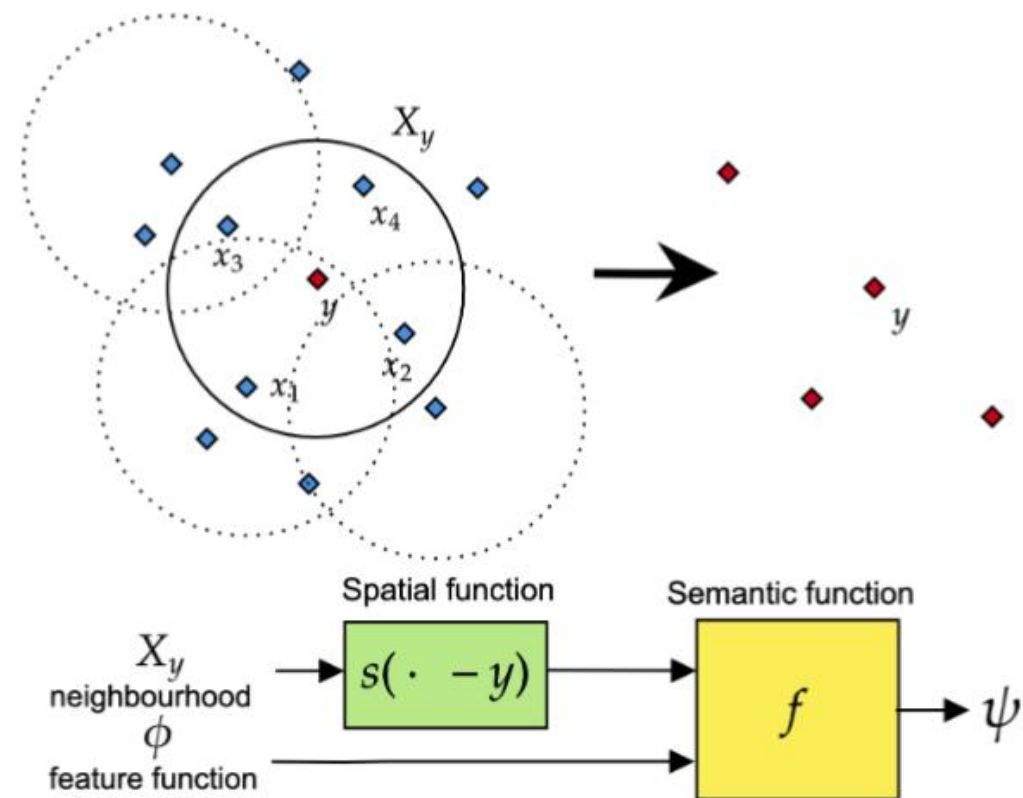
The detection time t^* is the first time instant when there is enough statistical evidence to claim that the datastream $\{x_1, \dots, x_t\}$ contains a change point.

A fundamental issue in change detection is to define a sequence of thresholds $\{h_t\}_t$ to control false alarms. We measure false alarms by the Average Run Length [2], defined as $ARL_0 = \mathbb{E}[t^*]$, where the expectation is taken assuming that the whole datastream is drawn from ϕ_0 . Thus, the ARL_0 is the average time before a false alarm. Ideally, the ARL_0 of an online change-detection method should be set *a priori*, similarly to Type I error probability in hypothesis testing.

3. Problem Formulation

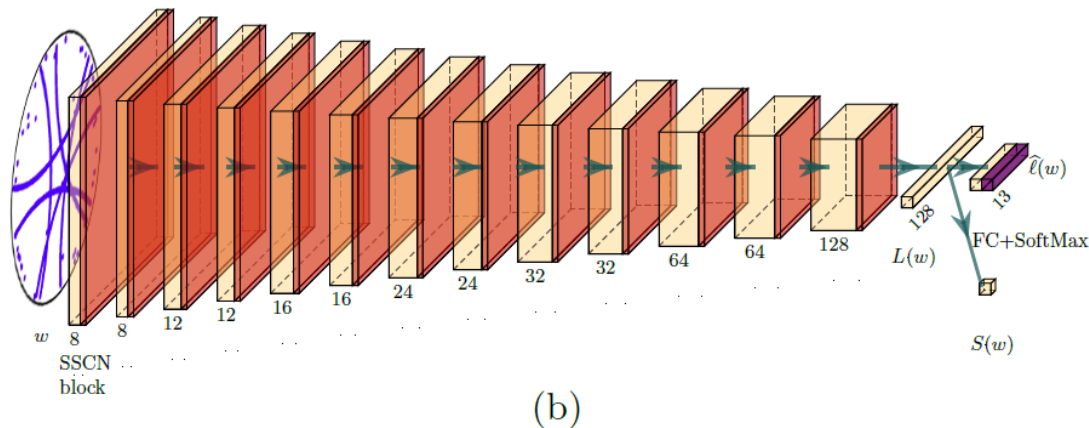
We denote as *point cloud* (PC) a pair (P, ϕ) , where $P \subset \mathbb{R}^d$ is a set of points in a d -dimensional space (in our case $d = 3$), and $\phi : P \rightarrow \mathbb{R}^I$ is a function associating a feature vector $\phi(x) \in \mathbb{R}^I$, *e.g.* a color, to each point $x \in P$. Our goal is to design a new layer that takes as input (P, ϕ) and separately processes P and ϕ , returning another PC (Q, ψ) , where $Q \subset \mathbb{R}^d$ and $\psi : Q \rightarrow \mathbb{R}^J$. The new layer has to enable a flexible design, where we can independently set the structure and number of parameter of the components handling the spatial information and the features.

We are particularly interested in deep NNs for unsupervised anomaly detection, which are exclusively trained on normal instances and detect anomalous PCs in the test set. This setup is also known as *semi-supervised* since only normal instances are provided for training.



3. Problem Formulation

A WDM w is a list containing the 2-dimensional coordinates indicating where defects lie within a wafer. These coordinates belong to a huge grid defined by the resolution of the inspection machines. For pattern recognition purposes, w can be seen as a binary image $w \in \{0, 1\}^{K \times K}$, where each pixel (i, j) corresponds to an inspected wafer location and $w(i, j) = 1$ if the coordinates (i, j) belong to the list and 0 otherwise. In normal production conditions, defects are rare and randomly distributed within the wafer, whilst failures might cause defects arranged in patterns like those shown in Figure 1(b). These patterns might either belong to a known class $\ell \in \mathcal{L}$ indicating a known production problem, or indicate a novel issue that had never been observed before. Although these previously unseen instances are usually referred to as *unknown* [32] or *out-of-distribution* [33] in the open-set recognition literature, we indicate them as *novelties* to highlight that these correspond to new defect patterns.



Description of what Wafer Defect Maps are, of the classification and open set recognition problems involved

Our goal is to train an open-set classifier $\mathcal{K}$ that associates to each WDM w either a known class label or the *Novel* label, which identifies WDMs containing novel types of patterns, i.e., the output of $\mathcal{K}$ is:

$$\mathcal{K}(w) = \begin{cases} Novel \\ \hat{\ell}(w) \in \mathcal{L}. \end{cases} \quad (1)$$

To this aim, we assume that a training set formed by n annotated WDMs $\mathcal{W} = \{(w_1, \ell_1), \dots, (w_n, \ell_n)\}$ – where each $\ell_i \in \mathcal{L}$ is a known label – is provided.

A major challenge of handling WDMs is that state-of-the-art deep learning methods such as Convolutional Neural Networks cannot be directly applied due to the huge size of the images obtained from full-resolution WDMs: in our case, $K = 20,000$, so a WDM would require almost 3 GB to be loaded in memory in single precision as a grey-scale image. A second challenge is the extreme class imbalance: indeed, the vast majority of WDMs belong to the *Normal* class, while some patterns, such as *BasketBall*, occur very rarely and are under-represented in the ST dataset, as shown in Figure 1(b).

Sometimes many ingredients need to be
presented...

come heuristic in nature (Kuncheva, 2013; Ditzler & Polikar, 2011) preventing a proper control over the FPR. Histograms, which are perhaps the most natural candidates for describing densities, enable a different form of monitoring that is based on a comparison among distributions (Ditzler & Polikar, 2011; Boracchi et al., 2017). However, they are often implemented over regular grids and require a number of bins that grows exponentially with the data dimension. Only a few change-detection solutions (Dasu et al., 2006; Boracchi et al., 2017) adopt alternative partitioning schemes that scale well in high dimensions. In particular, kqd-trees (Dasu et al., 2006) were introduced as a variant of kd-trees (Bentley, 1975) to guarantee that all the leaves contain a minimum number of training samples and have a minimum size. In (Boracchi et al., 2017) it is shown that histograms built on uniform-density partitions rather than regular grids provide superior detection performance.

Our main contribution is QuantTree, a recursive binary splitting scheme that defines histograms for change-detection purposes. The most prominent advantage of using QuantTree is that the distribution of any statistic defined over the resulting histograms does not depend on ϕ_0 . This implies that decision rules to be used in multivariate change-detection problems do not depend on the data, and can be numerically computed from synthetically generated univariate sequences. Moreover, histograms defined by QuantTree can have a pre-assigned number of bins and can be represented as a tree, thus enabling a very efficient computation of test statistics.

QuantTree (Section 3) iteratively divides the input space by means of binary splits on a single covariate, where the cutting points are defined by the quantiles of the marginal distributions. This splitting strategy is similar to the one adopted by kd-trees (Bentley, 1975), where the split is performed w.r.t. the median value of the marginal. Such a simple construction scheme can be handled analytically, as it is possible to prove (Section 4) that the distribution of each bin probability does not depend on ϕ_0 . Our experiments (Section 5) show that QuantTree enables good detection performance in high dimensional streams. Moreover, when testing few samples, QuantTree guarantees a better FPR control than the Pearson goodness-of-fit test and tests based on empirical thresholds computed through bootstrap. We also show that histograms constructed with a few bins gathering the same density under ϕ_0 achieve higher power than monitoring schemes based on different histograms.

2. Problem Formulation

Before the change, namely in stationary conditions, data in the monitored stream $\mathbf{x} \in \mathbb{R}^d$ are independent and identically distributed (i.i.d.) realizations of a continuous random vector $\mathbf{X}_0$ having an unknown probability density function

(pdf) ϕ_0 , whose support is $\mathcal{X} \subseteq \mathbb{R}^d$. We assume that a training set $TR = \{\mathbf{x}_i \in \mathcal{X}, i = 1, \dots, N\}$ containing N stationary data (i.e., $\mathbf{x}_i \sim \phi_0$) is provided.

Histograms: we define a histogram as:

$$h = \{(S_k, \hat{\pi}_k)\}_{k=1, \dots, K}, \quad (1)$$

where the K subsets $S_k \subseteq \mathcal{X}$ form a partition of $\mathbb{R}^d$, i.e., $\bigcup_{k=1}^K S_k = \mathbb{R}^d$ and $S_j \cap S_i = \emptyset$, for $j \neq i$, and each $\hat{\pi}_k \in [0, 1]$ corresponds to the probability for data generated from ϕ_0 to fall inside S_k . Both the subsets $\{S_k\}_k$ and probabilities $\{\hat{\pi}_k\}_k$ can be adaptively defined from training data TR , and in particular $\hat{\pi}_k$ is typically estimated as $\hat{\pi}_k = L_k/N$, i.e. the number of training samples L_k belonging to S_k over the number of points in TR .

Batch-wise monitoring: for the sake of simplicity, we analyze the incoming data in batches $W = \{\mathbf{x}_1, \dots, \mathbf{x}_\nu\}$ of ν samples. We detect changes by an hypothesis test (HT) which assesses whether data in W are consistent with a reference histogram h learned from TR . In particular, this hypothesis test can be stated as follows:

$$H_0 : W \sim \phi_0 \quad \text{vs} \quad H_1 : W \sim \phi_1 \neq \phi_0 \quad (2)$$

where ϕ_1 represents the unknown post-change distribution. We focus on HTs that are based on a test statistic $\mathcal{T}_h$ defined over the histogram h , like for instance the Pearson statistic (Lehmann & Romano, 2006). Thus, $\mathcal{T}_h$ uniquely depends on $\{y_k\}_{k=1, \dots, K}$, where y_k denotes the number of samples in W falling in S_k . We detect a change in the incoming W when

$$\mathcal{T}_h(W) = \mathcal{T}_h(y_1, \dots, y_K) > \tau, \quad (3)$$

where $\tau \in \mathbb{R}$ is a threshold that controls the FPR, namely the proportion of type I errors (Lehmann & Romano, 2006).

Goal: our goal is two-fold, *i*) learn a histogram h from TR to be used for change-detection purposes and *ii*) for each given test statistic $\mathcal{T}_h$ and reference FPR value α , define a threshold τ such that

$$P_{\phi_0}(\mathcal{T}_h(W) > \tau) \leq \alpha, \quad (4)$$

where P_{ϕ_0} denotes the probability under the null hypothesis that W contains samples generated from ϕ_0 .

There are two important comments. First, while (3) might seem an oversimplified monitoring scheme, this is enough to demonstrate that when histograms are built through QuantTree, the monitoring can be performed independently of ϕ_0 . As a consequence, test statistics $\mathcal{T}_h$ can be potentially employed in sequential monitoring schemes like (Ross & Adams, 2012). Second, we focus on general-purpose tests, which are able to detect any distribution change $\phi_0 \rightarrow \phi_1$ as well as on histograms that can model densities in high dimensions, i.e., $d \gg 1$.

1+ column for a 7
pages paper... but it
was needed to focus
on the specific
problem we solved

Aggregated features are stacked with the transaction data in the feature vector.

C. Supervised Information

Investigators' feedbacks are the most recent supervised information made available to the FDS, but represent only a small fraction of the transactions processed every day [20]. Additional labeled transactions are provided by cardholders that directly dispute unauthorized transactions [20], [63]. The timing of disputed transactions can vary substantially, since cardholders have different habits when checking the transcript of credit card sent by the bank. Moreover, checking disputed transactions entails some necessary administrative procedures that might introduce substantial delays.

All other transactions remain unlabeled: these can be either genuine transactions or frauds that were missed by the FDS and ignored by the cardholders. However, after a certain number of days have passed without cardholder dispute, all the unreported transactions are considered genuine by default, and inserted in the training set of the DDM.

Overall, there are two types of supervised information: *i*) feedbacks provided by investigators which are limited in number but refer to recent transactions, and *ii*) delayed supervised transactions, which are the vast majority for which the labels become available after several days (e.g. one month). This latter includes both disputed and non-disputed transactions.

D. System Update

Customers' spending behavior evolves and fraudsters continuously design new attacks, thus their strategies also change over time. It is then necessary to constantly update the FDS to guarantee satisfactory performance. Expert-driven systems are regularly updated by investigators who add ad-hoc (transaction-blocking or scoring) rules to counteract the onset of new fraudulent activities and remove those rules liable of too many false alerts. However, investigators can not modify the DDM, since it is not interpretable and can be only updated (e.g. re-trained) on the basis of recent supervised information, as shown in Figure 1. This operation typically requires a large number of labeled transactions, therefore, though investigators steadily provide feedbacks during the day, the classifier is usually updated/re-trained only once, notably at the end of the day, when a sufficient number of feedbacks is available.

III. PROBLEM FORMULATION

Here, we model the classification problem to be addressed in a real-world FDS, providing a formal description of the alert-feedback interaction and presenting suitable performance measures. The proposed learning strategy (Section V) and our experiments (Section VI) are built upon this model.

Let x_i denote the feature vector associated with the i -th authorized transaction and $y_i \in \{+, -\}$ be the corresponding class, where $+$ denotes a fraud and $-$ a genuine transaction. To cope with the time-variant nature of the transaction stream, a classifier $\mathcal{K}$ is updated (or newly retrained) every day. In particular, we denote by $\mathcal{K}_{t-1}$ the classifier that is trained on

supervised transactions available up to day $t-1$. The classifier $\mathcal{K}_{t-1}$ is then used to process the set of transactions T_t that have been authorized at day t . We denote by $\mathcal{P}_{\mathcal{K}_{t-1}}(+|x_i)$ the posterior of $\mathcal{K}_{t-1}$, namely the probability for x_i to be a fraud according to $\mathcal{K}_{t-1}$. Investigators check only few, high-risk, transactions. Thus, we model *alerts* as the k -most risky transactions, namely

$$A_t = \{x_i \in T_t \text{ s.t. } r(x_i) \leq k\}, \quad (1)$$

where $r(x_i) \in \{1, \dots, |T_t|\}$ is the rank of x_i according to $\mathcal{P}_{\mathcal{K}_{t-1}}(+|x_i)$, and $k > 0$ is the maximum number of alerts that can be checked by investigators². As discussed in Section II-A5, investigators contact the cardholders and provide supervised samples to the FDS in the form of *feedbacks*. In particular, feedbacks include all recent transactions from the controlled cards, which we model as:

$$F_t = \{(x_i, y_i) \text{ s.t. } x_i \text{ is from cards}(A_t)\}, \quad (2)$$

where $\text{cards}(A_t)$ denotes the set of cards having at least a transaction in A_t . The number of feedbacks, i.e., $|F_t|$, depends on the number of transactions associated with the k controlled cards. After a certain verification latency, the labels of all the transactions are provided to the FDS, since, as discussed in Section II-C, non-disputed transactions are considered genuine. For the sake of simplicity, we assume a constant verification latency of δ days, such that at day t the labels of all the transactions authorized at day $t-\delta$ are provided. We refer to these as *delayed supervised samples*:

$$D_{t-\delta} = \{(x_i, y_i), x_i \in T_{t-\delta}\}. \quad (3)$$

Note that $F_{t-\delta} \subset D_{t-\delta}$ since transactions at day $t-\delta$ obviously include those that have been alerted. Figure 2 illustrates the different types of supervised information available in a FDS.

It is worth mentioning that, despite our formal description includes several aspects and details that have been so far ignored in the fraud-detection literature, this is still a simplified model. In fact, alerts in a real-world FDS are typically raised online while transactions are being processed, without having to rank all transactions in T_t . Similarly, the delayed supervised couples do not come all-at-once, as each disputed transactions might take less (or possibly more) than δ days. Notwithstanding, we deem that our formulation takes into account the aspects of a real-world FDS that are the most important ones from a learning perspective, which include alerts, alter-feedback interaction and verification latency. We further comment that in principle, since the classifier analyzes each feature vector x_i independently, it does not alert cards receiving several risky transactions until any of these enters in the pool of the alerts (1). However, these situations are particularly relevant for investigators, and can be handled either by: *i*) suitable scoring rules or *ii*) feature augmentation, adding for instance a component that keeps track of the scores of recent transactions.

Fraud-detection performance can be conveniently assessed in terms of the *alert precision* $P_k(t)$, which is defined as:

$$P_k(t) = \frac{|TP_k(t)|}{k} \quad (4)$$

²Throughout the paper we denote by $|\cdot|$ the cardinality of a set.

where $TP_k(t) = \{(x_i, y_i) \text{ such that } x_i \in A_t, y_i = +\}$. Thus, $P_k(t)$ is the proportion of frauds in the alerts A_t . Though the classifier independently process each feature vector, the alert precision would be more realistically measured in terms of cards rather than authorized transactions. In fact, multiple transactions in A_t from the same card should be counted as a single alert, since investigators check all the recent transactions when contacting cardholders. This implies that k depends on the maximum number of cards that the investigators can control. In this context, it is more informative to measure the detection performance at the card level, such that multiple fraudulent transactions from the same card count as a single correct detection. Thus, we introduce CP_k , the *card precision*, as the proportion of fraudulent cards detected in the k cards controlled by the investigators:

$$CP_k(t) = \frac{|C_t^+|}{k}, \quad (5)$$

where C_t^+ denotes the set of fraudulent cards correctly detected at day t , namely, fraudulent cards having reported at least one alert. To correctly account for those days where less than k cards are fraudulent, we define the *normalized* $CP_k(t)$ as:

$$NCP_k(t) = \frac{CP_k(t)}{\Gamma(t)} \quad \text{with} \quad \Gamma(t) = \begin{cases} 1 & \text{if } \gamma_t \geq k \\ \frac{\gamma_t}{k} & \text{if } \gamma_t < k \end{cases} \quad (6)$$

where $\Gamma(t)$ is the maximum value of $CP_k(t)$ and γ_t is the number of fraudulent cards at day t . From (6) we have that $NCP_k(t)$ takes values in the range $[0, 1]$, while $CP_k(t)$ is in $[0, 1]$ when $\gamma_t \geq k$ and in $[0, \frac{\gamma_t}{k}]$ otherwise. For example, if at day t we have correctly detected 40 fraudulent cards ($|C_t^+| = 40$) out of the $k = 100$ cards checked by investigators, and the overall number of fraudulent cards is 50 ($\gamma_t = 50$), then $CP_k(t) = 0.4$ while $NCP_k(t) = \frac{0.4}{0.5} = 0.8$.

Note that, since $\Gamma(t)$ does not depend on the specific classifier $\mathcal{K}_{t-1}$ adopted, when the algorithm "A" is better than algorithm "B" in terms of CP_k , "A" is also better than "B" in terms of NCP_k . Moreover, because of verification latency, the number of fraudulent cards in day t (i.e., γ_t), can be only computed after few days, therefore NCP_k cannot be computed in real time. Thus we recommend using CP_k for assessing the running performance, while NCP_k for backtesting e.g. when testing different FDS configurations, as in Section VI-F.

PF describing a real world fraud detection setup was claimed as a contribution (and followed a text describing how fraud detection is being performed in a bank)

Concluding Remarks

Problem Formulation and Applications

You might be addressing a very specific application in your thesis

- *e.g. learning data-driven models describing the heartbeats of a specific user, and adapting these when the heart rate changes*

PF is then good! The modeling underpinning PF is already a contribution... inputs and outputs and the operating conditions need to be described

You might address a more general problem, which touches on multiple applications. Still, the input, output, goals, and rules of the game have to be clearly described.

Very general problems (*e.g., clustering*) might necessarily remain rather broad in their problem formulation

PF and DL

Deep learning does not help! Since solutions became black boxes you might be tempted to treat your problem as well as a black box!

Probably PF for algorithms based on a solid mathematical / statistical modeling are easier to describe

... however, «rules of the game» are particularly important for problems being addressed by DL models... you might even take some inspiration from additional research directions from PF

3.1 Observation model

We consider observations as noisy grayscale images $z : X \rightarrow \mathbb{R}$ that can be modeled as

$$z(x) = y(x) + \eta(x), \quad x \in X \subset \mathbb{Z}^2, \quad (1)$$

where $X \subset \mathbb{Z}^2$ is the image domain, which is assumed a regular pixel grid, $y : X \rightarrow \mathbb{R}$ is the unknown original image, and $\eta : X \rightarrow \mathbb{R}$ is i.i.d. Gaussian white noise, $\eta(\cdot) \sim \mathcal{N}(0, \sigma^2)$.

For the sake of simplicity and notation, we assume that images can be extended beyond the boundary of X to the whole $\mathbb{Z}^2$ through any standard padding technique¹.

Problem formulation is needed to ...

- Understand your problem and gain a sufficient generalized insight
- Talk to others about your problem
- Identify «cognate problems» or other solutions that in a different domain are addressing the same problem of you
- Gain insight and possibly intuitions for a better solution
- Adjust, change, and modify the problem you are addressing if something different becomes more interesting
- Graduate, it has to be a chapter in your thesis